Episodio 5 de 14: Cómo se hizo la réplica, y cuánto fiarse

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 4, «La solución del paper: el taco de vales».
- **Episodio siguiente:** episodio 6, «Simular 4 procesadores con un solo PC».

Lo esencial de este episodio:

- Se usaron los programas oficiales del artefacto de Zenodo, con su MD5 comprobado: es el código de los autores, no una copia.
- Adaptaciones: los hilos se dividieron entre 6 (24 son toda la máquina, como 144 en el paper) y se midieron 5 rondas de 2 s, en orden aleatorio, en vez de 7 de 10 s.
- El recorte se validó con la receta exacta: sin Docker se desvía de −10 % a +19 %, y el orden se mantiene en 4 de 6 escenarios. Por eso las diferencias pequeñas se tratan como empate.

Parte 4. Cómo se hizo la réplica

4.1 Qué hicieron los autores

El paper mide dos cosas:

- **Rendimiento (figura 5)** en una máquina de **4 Intel Xeon E7-8867: 72 núcleos, 144 hilos** de hardware. Se crean *n* pilas concurrentes. Cada operación elige una pila al azar: con probabilidad *u* % saca un elemento y lo mete en otra pila, y si no, lee la cima. Se varía el número de hilos, en 6 combinaciones de (n, u).
- **Latencia (figura 6)** en un **Xeon w7-3465X: 28 núcleos, 56 hilos**. *p* hilos a la vez: el primero se mide y los demás hacen `store` sin parar. Se miden `load`, `store` sin liberar y `store` que libera.

Interpretar n y u es fácil: **n pequeño significa mucha gente en poco sitio**, y **u grande significa mucha escritura**. Con n = 1 todos se pelean por el mismo tablón; con n = 10^6 casi nunca coinciden.

4.2 Qué se usó aquí

Para que la comparación sea justa se usaron **los binarios oficiales**, compilados desde el artefacto publicado en Zenodo (`10.5281/zenodo.11775182`, MD5 comprobado), en la misma imagen base: Ubuntu 22.04 con g++-12. Es decir: el mismo código de los autores, el de Folly y el de Anderson, y el mismo programa de pruebas. No es una reimplementación.

También se corrió `codigo/bench.cpp` del equipo, como comprobación independiente. Son dos programas distintos que deberían contar la misma historia. La Parte 6.6 compara lo que dicen.

Del artefacto se extrajo la receta exacta, que no aparece en el paper:

Detalle	Valor
Orden	`bench_stack -r 10 -s n -u u -i 7 --peek=true --threads=T --alg=A`
Algoritmos	`ivo` (Autores), `gnu` (Libstdc++), `jss-free` (Williams), `folly`, `mine` (Anderson)
Asignador	`jemalloc` con `LD_PRELOAD`
Memoria	`numactl -i all`
Hilos	1, 18, 36, 54, 72, 108, 144, 180, 216
Tamaño de pila	20 elementos + T/n
Resultado	Mediana de 7 ejecuciones de 10 s

4.3 Adaptaciones, y por qué

Paper	Aquí	Por qué
Hilos 1 a 216	**1, 3, 6, 9, 12, 18, 24, 30, 36**	Es la lista del paper dividida entre 6. La proporción se conserva: 24 = «toda la máquina», como 144; 36 = 1,5 veces, como 216
7 × 10 s, en un mismo proceso	5 rondas de 2 s, cada punto en un proceso nuevo y en **orden aleatorio**	Reduce el tiempo de la figura 5 de unas 5 h a poco más de 1 h. Además, si la CPU se calienta o algo interrumpe, afecta a todos por igual
p = 56	p = 1, **24** y 56	24 son todos los núcleos reales. Con 56 habría más hilos que núcleos; se mide también, pero no es comparable
Williams no aparece en la figura 6	**Sí se midió**	El paper lo omite «porque su latencia casi siempre era mayor», y el artefacto trae esa línea comentada. Se activó esa línea para comprobar la frase, en vez de creerla, y se compiló con `-mcx16` en un binario aparte, porque sin esa bandera lleva un candado escondido (ver la Parte 6.2). Son los únicos cambios en las pruebas oficiales, además de la lista de valores de p

Y como recortar puede sesgar, se verificó: varios puntos se repitieron con la receta **exacta** de 10 s × 7 y se comparó (ver Parte 6.7).

4.4 De dónde salen los números del paper

Las figuras del PDF son vectoriales. No hay que leerlas a ojo: `herramientas/extraer_fig5.py` saca la posición exacta de cada marcador y la convierte a Mop/s usando la cuadrícula. El resultado está en `paper_fig5.csv`. Así se compara número contra número.

6.7 Validación de los recortes

Se midieron 2 segundos en lugar de 10. ¿Y si eso cambia el resultado? Para comprobarlo se repitieron dos escenarios con la receta **exacta** del paper: 7 ejecuciones de 10 s, con 24 hilos.

Escenario	Implementación	10 s × 7	2 s × 5	Diferencia
n = 1, u = 1 %	Autores	19,00	19,81	+4,3 %
	Anderson	16,99	18,64	+9,7 %
	Folly	7,53	9,11	+21,0 %
	Williams	4,29	4,83	+12,6 %
	Libstdc++	1,39	1,19	−14,8 %
n = 10^6, u = 50 %	Libstdc++	60,43	56,34	−6,8 %
	Anderson	58,69	56,50	−3,7 %
	Folly	57,70	53,39	−7,5 %
	Autores	56,30	55,37	−1,7 %
	Williams	42,44	40,39	−4,8 %

La lectura honesta tiene dos partes:

- **El recorte no es inocuo.** Una diferencia del 21 % no es ruido despreciable. En `n = 1, u = 1 %` el orden no cambió, pero los valores sí se mueven bastante. Las dos tandas se diferencian además en otra cosa: las 7 ejecuciones largas corren en un mismo proceso, las cortas en procesos nuevos.
- **Sirve para el orden, no para el decimal.** Con n = 1 el orden es idéntico. Con n = 10^6 cambian los cuatro primeros entre sí, pero están a menos de un 8 %: confirma que ese escenario es un empate, y que hay que leerlo como tal.

Sin Docker, en los 6 escenarios. Dos escenarios son pocos, así que la comprobación se repitió con los programas oficiales fuera de Docker, esta vez en los 6: 30 puntos, cada uno con 7 ejecuciones de 10 s. Las cifras están en la tabla «Validación de los recortes (nativo_oficial)» de `tablas.md`. Salen tres cosas:

- **Cuánto se desvía.** La mitad de los puntos se desvía menos de un 6,7 %. Pero 10 de los 30 se desvían un 10 % o más, y los dos peores son Folly con n = 10 y u = 10 % (+19,0 %) y Folly con n = 1 y u = 50 % (+18,8 %).
- **Se desvía casi siempre hacia el mismo lado.** Con contención (n = 1 y n = 10), la versión corta da **más** que la larga en 23 de 25 puntos. Con n = 10^6 da **menos** en los 5. Es como un cronómetro que adelanta en una pista y atrasa en otra: no es ruido al azar, es un sesgo. Por qué pasa no se midió. Recuerda que las ejecuciones largas comparten proceso y las cortas no; podría tener que ver, pero es una suposición.
- **El orden casi no cambia.** Es idéntico en 4 de los 6 escenarios. En n = 1 y u = 10 %, Autores y Folly se cruzan (4,38 frente a 4,24 Mop/s con la receta larga; 4,54 frente a 4,64 con la corta), pero los dos están a menos de un 4 %. En n = 10^6 los cuatro primeros cambian de puesto, y ya se sabía que ahí hay un empate.

Por eso en este documento no se dan por buenas las diferencias pequeñas. Si el recorte puede mover a un algoritmo un 19 % y a otro menos de un 1 %, una ventaja de un 10 % entre los dos

no demuestra nada; una de 2 o 3 veces, sí.
